Reviewer Pack — Minimal Index of Tropicalized Symplectic Leaves and DPLL Pro…

Entry 9f377a5314c3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 16:42:26 UTC

Minimal Index of Tropicalized Symplectic Leaves and DPLL Proof Width

Entry ID: `9f377a5314c3` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 16:42:26 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry × Symplectic Geometry **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

> For every Boolean satisfiability instance φ with resolution proof width w(φ), the minimal index of tropicalized symplectic leaves in its associated symplectic leaf system is linearly correlated with its DPLL proof width, such that the minimal index of tropicalized symplectic leaves = Θ(w(φ)).

Rationale (proposer's reasoning):

> Symplectic geometry provides a geometric structure on spaces of polynomials that has been previously applied to analyze the complexity of real polynomial systems. Tropicalization offers a bridge between algebraic and geometric structures. If this conjecture holds, it would indicate a deep connection between symplectic geometry and computational complexity by providing a lower bound for DPLL proof width in terms of a geometric invariant.

Taxonomy category: `symplectic_tropical_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d1dc237f5e6021f2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal index of tropicalized symplectic leaves equals resolution proof width within a factor of 2, measured by the ratio |minimal_index - w(φ)| ≤ 0.5 * w(φ), across at least 80% of instances with more than 20 variables.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry AND symplectic geometry AND DPLL proof complexity` - `resolution proof width AND tropicalization AND symplectic leaf system` - `Boolean satisfiability AND minimal index of tropicalized symplectic leaves AND θ(w(φ))`

Top relevant hits considered: - [http://arxiv.org/abs/1612.04764v1] Cohomological aspects on complex and symplectic manifolds - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/math/0403015v1] Amoebas of algebraic varieties and tropical geometry - [http://arxiv.org/abs/astro-ph/0208243v7] Measurement of the Flux of Ultra-high Energy Cosmic Rays from Monocular Observations by the High Resolution Fly's Eye Exp - [http://arxiv.org/abs/astro-ph/0407622v3] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye - [http://arxiv.org/abs/1511.06778v1] The 1st Fermi Lat Supernova Remnant Catalog - [http://arxiv.org/abs/math/0611768v5] The Invariant Symplectic Action and Decay for Vortices - [http://arxiv.org/abs/2103.16512v2] The tropical symplectic Grassmannian

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def resolution_width(formula):
        if ' and ' not in formula:
            return 1
        left, right = formula.split(' and ', 1)
        return max(resolution_width(left), resolution_width(right)) + 1

    def generate_formula(n):
        if n == 0:
            return random.choice(['True', 'False'])
        else:
            op = random.choice(['and', 'or'])
            left = generate_formula(n - 1)
            right = generate_formula(n - 1)
            return f'({left} {op} {right})'

    def tropical_index(formula):
        if formula == 'True':
            return 0
```

```
        elif formula == 'False':
            return math.inf
        else:
            left, op, right = formula[1:-2].split(' ')
            if op == 'and':
                return max(tropical_index(left), tropical_index(right))
            elif op == 'or':
                return min(tropical_index(left), tropical_index(right))

    n_max = 0
    instances_tested = 0
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        if n > n_max:
            n_max = n

        for _ in range(5):
            formula = generate_formula(n)
            w_phi = resolution_width(formula)
            t_index = tropical_index(formula)

            instances_tested += 1
            total_metric_value += abs(t_index - w_phi) / w_phi

            if abs(t_index - w_phi) > 0.5 * w_phi:
                conjecture_holds = False
                counterexample = f"Formula: {formula}, T-index: {t_index}, W-phi: {w_phi}"

    mean_metric_value = total_metric_value / instances_tested

    return {
        "metric_name": "Tropical Index vs Resolution Width Ratio",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, min(30, len(primes)))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
```

```
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"{results[next(i for i, r in enumerate(results)
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f23fcdb2.py", line 75, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f23fcdb2.py", line 45, in run_trial
    w_phi = resolution_width(formula)
            ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f23fcdb2.py", line 32, in resolution_wi
    left, right = formula.split(' and ')
    ^^^^^^^^^^^
ValueError: too many values to unpack (expected 2)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13825
2	preregistration	ollama_remote	glm4:latest	0	0	9239
3	novelty	ollama_remote	glm4:latest	0	0	8651
4	novelty	ollama_remote	glm4:latest	0	0	9729
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12450

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7959
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9140
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10936
9	judge	ollama_remote	glm4:latest	0	0	12698

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94626 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9f377a5314c3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9f377a5314c3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9f377a5314c3.tar.gz` (if generated)